ChatGPT

Pathfinder solver research memo

Executive judgement

The literature points to two different failure mechanisms in your persistent set, not one. The L92 pattern is an **obligation-plateau** problem: the search is not reducing mandatory structure fast enough, and the flat obligation-reduction slope is exactly what landmark and operator-counting work treats as a sign that the heuristic is not charging enough for still-unmet subgoals. The L108 pattern is a **final-plateau / closure-ranking** problem: the search gets into a huge shell of states with essentially the same lower bound, and then tie-breaking and local completion matter more than the base heuristic. L134 looks mixed: it has enough closure behaviour to benefit from a closer, but it also has enough blocked/portal structure that the explorer's lower bound is probably too weak. The bad news is that a single global scalar tweak is the wrong medicine. The good news is that the planning literature gives a fairly clean split: strengthen obligation handling for L92-like instances; add a bounded, exact-ish closer for L108-like instances; and keep those changes ring-fenced so they do not globally perturb the 134 already-working levels. [1]

The highest-value architectural fact in your whole description is that every level already ships with a verified hint path. In planning terms, that means you already possess a machine-checkable witness for solvability, a ranked sequence of states for heuristic diagnostics, and a ready-made counterexample source for unsafe pruning. That is unusually valuable. The strongest practical recommendation from the research base is therefore not "replace the stack"; it is "instrument the existing stack against the oracle you already own, then add the smallest stage-specific mechanisms that literature says are robust." [2]

One conclusion is especially firm: **Luby restarts without retained learning are not the next move here.** Restart theory is strong for randomized search with heavy-tailed runtimes, but the modern restart literature ties most of its wins to retained clauses, restart-aware branching, or at least meaningful between-run independence. In your deterministic browser beam with an existing attempt ladder, adding a bare Luby schedule is much more likely to add overhead than to create a new source of information. That part of the tech memo matches the literature. [3]

Obligation plateaus and obligation-aware heuristics

What the sign of must-pass urgency should mean

The canonical planning convention is simple: obligations that still remain unsatisfied are represented as a **non-negative residual**. Classic goal-count heuristics count missing goals; landmark-count heuristics count landmarks that still need to be achieved; LAMA's landmark heuristic counts not-yet-accepted or required-again landmarks and then combines that distance-style quantity with a cost estimate. In other words, "more unsatisfied mandatory things" conventionally means "heuristically worse," unless you explicitly define your search score as a maximization objective and subtract that residual as a penalty. A raw feature value that becomes more negative when an unmet must-pass remains unmet is therefore not intrinsically wrong, but it is only correct if the coefficient/sign convention elsewhere turns that into a lower priority. If your telemetry says the term is *pulling against* goal progress and the level times out with must-passes still

outstanding, the default prior should be “inverted gradient or incompatible weighting,” not “clever intentional detour bonus.” [4]

That also answers the “when is negative urgency intentional?” part. It is intentional only when the feature is not actually “urgency” but something like “detour risk,” “premature commitment penalty,” or “delay bonus” inside a staged policy. LAMA itself is costs-plus-distance, not a scheme that rewards leaving landmarks behind; and the mainstream goal-count / landmark-count family is very explicit that remaining obligations are counted as residual work. I do not know of a canonical planning heuristic where a mandatory fact residual is given the opposite gradient *by default* and then works well globally. [5]

The cleanest way to test intent versus sign bug without reverse-engineering the whole linear driver is to use the hint paths as an oracle and run three offline checks. First, **local monotonicity on the witness**: along each hint prefix, score deltas should correlate with decreasing residual obligations more often than not; if must-pass urgency systematically worsens on witness steps that move toward still-unmet must-passes, the sign is almost certainly wrong. Second, **dominance testing on sibling states**: if state A is no farther from the goal, has no less slack, and satisfies strictly more mandatory obligations than B, the score should never prefer B absent a very explicit staging rule. Third, **counterfactual re-scoring**: replay logged frontiers with only the must-pass term sign flipped; if the frontier ranking suddenly correlates much better with oracle depth or obligation completion while all other terms stay fixed, you have isolated the defect to sign/scale rather than global search structure. The ranking-based planning literature is useful here because it explicitly argues that what matters for forward search is state ordering, not faithful regression to cost-to-go. [6]

Held–Karp, landmarks, and must-cross edges

Using a Held–Karp waypoint lower bound on the *real* obligations is well grounded. What is not well grounded is treating “remaining required intersections” as ordinary virtual waypoints and then summing them naively into the tour. A self-intersection event is not a location that must independently be visited once; it is a global path-shape property that can be co-satisfied with movement that also reaches a must-pass, a must-cross, or the goal. If you encode each remaining intersection as an ordered virtual waypoint, you create a double-counting risk: the same future segment can pay for both geometry and obligation, but the tour bound will charge both unless you prove separability. The safe literature-backed move is to keep **waypoint/tour obligations** and **intersection obligations** as separate lower bounds and combine them by `max`, or by a cost-partitioned / operator-counting formulation that proves how much resource each component may charge. [7]

For must-cross edges, you do **not** need a dummy-vertex transformation as the primary encoding. The newer transition-landmark work is almost exactly what you want: it treats requirements that must hold in every solution of a transition system as transition-counting constraints, and it shows how cuts induce disjunctive action landmarks. In Pathfinder terms, a required edge traversal is naturally an edge/transition landmark or an operator-counting constraint such as “this directed or undirected edge must be used at least once.” That is much cleaner than inserting a degree-2 dummy vertex, because the dummy-vertex move perturbs `reqLen` by construction and forces you to redefine how intersections are counted around the inserted vertex. Unless you want to rebuild the whole semantics around a transformed graph, the direct transition-landmark/operator-counting encoding is the sounder route. [8]

On combining landmark count with a metric tour term, the core distinction is this: LAMA-style landmark count is **inadmissible and path-dependent**, whereas landmark-cost partitioning and operator-counting

combinations are admissible because they explicitly divide operator cost or unify valid constraints. There is no general theorem that a hand-tuned linear combination `α·landmarks_unmet + β·tsp_lb + γ·goal_dist` will be monotone-non-increasing along a known-good path. The admissible literature says: if you want guaranteed compatibility, use `max(...)`, a valid cost partitioning, or the union of operator-counting constraints. If you are doing satisficing search and care more about ranking than admissibility, the safest engineering move is not a single scalarized sum; it is a **lexicographic key** such as `(unmet mandatory obligations, tour LB, product-graph goal distance, slack)` so that a tiny weight mistake cannot globally invert priorities. [9]

Whether portal state changes the waypoint metric

If the portal commitment automaton changes which moves are legal or how a portal traversal behaves, then the relevant shortest paths live on the **product graph** `(cell, portal_state)`, not on naked cells. In that case, inter-waypoint distances are state-dependent, but they are dependent only on the finite automaton state, not on the full frontier node identity. So the literature-guided answer is: precompute shortest-path tables once per **(level, automaton-state)** product graph, or per `(waypoint, automaton-state)` source if that is cheaper. You should not recompute from scratch per frontier node unless something else in the state besides the portal automaton changes transition legality. This is the same product-automaton idea used in heuristic search for temporally extended goals and other finite-automaton path constraints. [10]

The cleanest joint feasibility bound

The reverted bound `intersectionDeficit + minDistToGoal > rSteps` is exactly the sort of thing that can become unsafe once obligations can overlap in the same suffix. The literature does not give a tidy closed form for your exact puzzle semantics, but it gives a clean design principle: **sum only independent resource lower bounds; otherwise take max or use a joint optimization relaxation**. In practice, the nicest principled approximation here is an operator-/transition-counting LP on the residual state: add one constraint for geodesic reachability in the current product graph, landmark/transition constraints for each must-pass and must-cross requirement, and a separate family of constraints for minimum extra transition usage needed to realize the remaining intersections. Then let the LP compute the tightest lower bound without double-counting. If you do not want that implementation yet, the downgraded version is `max(h_goal, h_waypoints, h_intersections)` plus a separate slack check, not a naive scalar sum. [11]

Final-mile plateaus and closers

Why a solver can see thousands of states at lower bound one and still fail

The short answer is: because **being one step from validity is not the same thing as telling the search which of those one-step states is the right shell to enter**. The A* tie-breaking literature is very clear that once a search enters a large final `f`-layer or “final plateau,” the secondary ordering can dominate runtime. Corrêa, Pereira, and Ritt sharpen this further with the distinction between nonpathological spaces, where tie-breaking barely matters, and pathological spaces, where tie-breaking determines how much of the final plateau gets explored. Asai and Fukunaga show that this is not a corner case: large plateaus are common,

and standard “break ties on smaller h” folklore is often not enough. L108’s `lb = 1` with 2,500–15,000 states is textbook plateau behaviour. [12]

The right diagnostic split is therefore three-way. If almost all `lb = 1` states get the same secondary score, you have **heuristic insensitivity** in the closure shell. If changing only tie-breaking or FOCAL ordering changes success materially, you have **tiebreak degeneracy**. If replaying the verified hint path shows that the actual legal closure move is getting rejected by a supposedly soft prune, then the closer problem is actually **pruning too hard**. The literature on enforced hill-climbing is also relevant here: Hoffmann and Nebel’s explanation of plateau escape is exactly that once evaluation flattens, a short complete search for a better state can be the difference between progress and permanent wandering. [13]

The best secondary cost for FOCAL-style rescue

For your closure regime, the most defensible secondary cost is **distance-to-go / residual-violation distance**, not a second cost-to-go estimate. That is the line running from `A*ε` through Explicit Estimation Search: use a lower bound only for admissibility or boundedness, and use a separate estimate of *distance to a satisfactory goal* to decide which node inside the acceptable shell should be expanded next. `A*ε` using plain `d` has known pathologies because low-`d` nodes can have high `f` and keep children out of FOCAL; EES fixed part of that by combining an admissible lower bound with inadmissible cost and distance estimates. Later bounded-suboptimal work keeps the same theme: within the acceptable set, sort by closeness to a solution, probability of yielding a feasible solution, or expected effort. [14]

Translated to Pathfinder, the secondary key should be a closure-specific residual metric, something like:

`d_close(s) = (unmet required visits + unmet must-crosses, |len deficit|, |int deficit|, goal distance in product graph, portal mismatch)`

ordered lexicographically or converted to a tiny integer rank. If you want a single scalar, a dynamic-potential style score is also defensible: among nodes inside the bounded shell, prefer the one with the highest estimated probability of becoming fully feasible under the remaining length budget. But in implementation terms, an EES-style `closest valid closure first` policy is the smallest lift and the most directly supported by the literature. [15]

How deep a bounded closer can go in browser JavaScript

Using your own stated envelope of roughly 10 million primitive ops per second and a branching factor around 6–8 in portal-active states, the combinatorics get ugly fast. A plain breadth-first tree of depth 6 is about 56k nodes at branching 6, 137k at branching 7, and 300k at branching 8. Depth 7 rises to about 336k, 961k, and 2.4M respectively. Depth 8 rises again to about 2.0M at branching 6 and 19.2M at branching 8. That makes **depth 6** the defensible default for a 50 ms closer, with **depth 7** only tolerable if you aggressively prune, start from very few seeds, or have an effective admissible residual bound that collapses the tree. Depth 8 is not a sane browser-default in the wide-branch case.

For exact-length closures, the literature does favour a bounded-cost or best-first closer over raw BFS. Recent bounded-cost search work explicitly argues that an absolute cost bound can simplify things relative to a moving `f_min`, and bounded-cost EES sorts the candidate shell by distance-to-go rather than just depth. That is much closer to your `reqLen` regime than pure BFS, which treats all depth-`k` prefixes alike

even when only a narrow band of remaining-length balances is actually viable. So if you build a closer, it should be a **bounded-cost best-first / EES-lite closer**, not a literal unweighted BFS except as a tiny fallback. [16]

Caching the closer's explored set across seeds is sound **only** when the cache key includes every state component that affects future feasibility: position, portal automaton state, remaining length budget, remaining intersection budget, and the residual mandatory-obligation mask at minimum. If you omit any suffix-relevant field, the cache stops being a sound completion-explored set and becomes, at best, a heuristic memo. In your domain, reusing dead-end status across different residual `reqLen` or `reqInt` values is especially dangerous. That is not a subtle theorem from the literature; it is the direct consequence of running search on a non-Markovian projection. [17]

Why meet-in-the-middle is probably not the first browser closer to build

Holte et al.'s MM and the must-expand-pairs line assume a clean backward search over reverse operators. If the portal automaton is represented so that predecessor enumeration is well-defined on the full product state, nondeterministic predecessor *sets* are fine; backward search does not require a unique predecessor. The real problem is elsewhere: `reqInt` and some closure legality are global path properties, so a backward state that forgets the right history is not Markovian. In that reduced state space, the reverse operator is not truly defined, and a bidirectional theorem no longer applies. [18]

That means a portal-aware MM is only sound if you backward-search the full product of geometric state, automaton state, and every residual quantity that determines future legality. That is a lot of machinery for browser JS. Recent bidirectional work broadens the theory and tightens search bounds, but it still presumes you can honestly define the backward problem. Given your constraints, a bounded-cost closer is a much better two-week target than MM, BAE*, or PEM-BAE*. [19]

Rescue gates and regression-safe engineering

Gates should be treated as classifiers, not sacred conjunctions

The robust solver-engineering pattern here is: **log every subpredicate, evaluate them against historical outcomes, and treat the gate as a calibrated classifier over failure modes**. Dynamic restart work is relevant because it explicitly uses real-time observations about solver behaviour rather than a fixed blind schedule. The right lesson is not "always use a sliding window," but "if you have measurements, use them explicitly." So yes: record per-predicate truth values for every attempt; yes: compute which predicates actually separate rescue-worthy runs from ordinary timeouts; and only then decide whether the fire rule should stay conjunctive, become staged, or become a `k-of-n over recent attempts` policy. [20]

In your current situation, the hard evidence already says some of the proxy counters are wrong for the relevant failure families. When a counter never reaches threshold on the very class it was designed to catch, the principled fix is **not automatically to lower the threshold**. The principled fix is to ask whether the counter has any predictive power. If it does, lower or smooth it. If it does not, replace it with a derivative/stall feature that correlates with the failure mode. The most useful methodology language here comes from ablation and parameter-importance analysis: isolate which signals matter, and distinguish necessary mechanisms from spurious configuration side effects. [21]

For L108-like closure flooding, a sliding-window criterion is actually literature-consistent so long as you base it on predictive signals, not on aesthetic symmetry. A gate such as “activate closer if `lb <= 1` persists across multiple attempts and near-solution shell size remains high” is much closer to a dynamic restart policy than a one-shot conjunction with a brittle frontier threshold. For L92-like stagnation, the better gate is a **stalled obligation-reduction slope** or **stalled must-cross progress** discriminator, because that is the observable failure, not “frontier contains at least N states of a special named kind.” [22]

Replay the hint paths before merging any new prune or bound

What you keep proposing informally is exactly the right idea. Before any pruning or bound change lands, **replay every known-good hint path through the new state model and assert that no hard prune fires on any prefix**. In classical planning, the nearest standardized tools are plan validation and certifying algorithms: planners already use validators such as and INVAL to check that a purported solution really is valid, and the certifying-planning line treats witnessed outputs as objects that should be independently checkable. That is not identical to your use case, but it is the right intellectual bucket. If a new prune rejects a known-valid witness prefix, the rule is wrong *for deployment purposes*, regardless of whether the instance still happens to solve by a different route. [23]

The closest published name for the broader pattern is “counterexample-guided refinement.” Seipp and Helmert’s CEGAR work refines abstractions by taking an abstract counterexample, checking why it fails concretely, and refining so the same failure does not recur. Your version is simpler and more practical: the hint path gives you concrete counterexamples to unsafe pruning and bad heuristic ranking. That makes “counterexample-guided prune refinement” a fair label even if it is not a canonical competition term. [24]

The rollout pattern is also clear. First, ship new bounds in **shadow mode**: compute them, log when they would prune, but do not enforce. Second, run hint-path replay and diff every formerly solved level against the current branch. Third, use ablation-style analysis to identify which exact rule, threshold, or coefficient caused the regression rather than reverting a whole PR bundle blind. Fawcett and Hoos’ ablation analysis is not about planning per se, but it is exactly about separating necessary modifications from spurious side effects in high-dimensional configuration changes. [21]

Keep telemetry out of the canonical solve result

The identity-stamp regressions have an easy architectural answer: the canonical solve result should be **immutable semantic output only**. Attempt-level metadata belongs in a separate telemetry sink keyed by `(level_id, run_id, attempt_ordinal)` or equivalent. That is the same general separation used by mainstream solvers: search logs and statistics are diagnostic channels, not part of the returned plan/solution object. The official documentation around and external guidance on its `log_search_progress` parameter treat progress metrics as logs; the similarly expose search configuration and statistics separately from the plan result. [25]

Given your history, I would make this a hard architectural boundary: the audit equality comparator must never see attempt seed, attempt count, policy profile, overlap telemetry, or branch-correlation metrics. Any future work on portfolio diversity should write to a stand-alone run journal and then join on run ids after the fact. That is low drama, high leverage, and directly targets one of your highest-frequency regression classes. [26]

Browser-only exact solving and adaptive policy

What browser-side CP/SMT looks like in practice

The current browser story is good enough for tooling and occasional fallback, but not good enough to make a general-purpose exact solver the default answer under your stated payload and latency budget. The official shows browser-capable JavaScript bindings, and the online guide literally runs examples in-browser. The official supports browser use via WebAssembly, and the MiniZinc playground runs locally in the browser. The official points to an in-browser runner, and community builds like `clingo-wasm` do work. But official MiniZinc build docs still call the WebAssembly support experimental, community Z3 WASM builds report slow loading, and recent browser-packaging examples for Z3 still show very large WASM artefacts. [27]

That makes the answer to your practical question fairly blunt. For a single-threaded browser target with a **≤5 MB** payload and **sub-500 ms** turnaround, the state of the art does not support relying on a stock general-purpose SMT/CP engine as the default shipped solver for this puzzle class. MiniZinc-in-browser is real but positioned openly for browser use and prototyping; `clingo-wasm` is real but again primarily a portability layer; a community `cpsat-js` package now exists, but there is still no clear official browser deployment path for CP-SAT comparable to the native APIs. Z3 works in-browser, but current packaging examples make the bundle-size problem obvious. I did not find credible public evidence that any of these, as shipped today, reliably hit the "~100 ms class" on a product-state path puzzle under your payload cap. [28]

So the browser-safe blueprint is the hybrid one you already sketched. Ship the precomputed full-solution witnesses for the fixed 137 shipped levels; use the live JS solver for partial-state hinting; and only if needed, add a **tiny exact micro-solver for bounded suffixes**, not a full general-purpose solver stack. From a software-risk perspective, that is much closer to the certifying-algorithm model: small witness, small validator, small targeted exact search. [29]

Nogood and trap learning in JS

The planning literature does support trap and nogood learning as a way to detect dead ends and avoid repeating them. Lipovetzky, Muise, and Geffner formalize traps as sets closed under transitions and show how they capture dead-end formulas; Steinmetz and Hoffmann refine dead-end detectors online by learning conjunctions when search encounters missed dead ends. That is enough to justify a small projected-state nogood learner in Pathfinder. [30]

What I did **not** find is a published implementation study that answers your exact GC question for JavaScript. There does not appear to be a canonical "above X nogoods, JS overhead outweighs the win" threshold in the literature. Given that absence, the engineering answer should be conservative: use packed integer projections, not object-heavy structures; keep nogoods tiny; and store them in flat maps or typed buffers if possible. For your proposed projection `(tile_position, portal_state, obligations_mask)` with only about a dozen obligation bits, a packed 32-bit or 53-bit integer key is entirely reasonable. A `Map<number, number>` or `Map<number, Uint32Array-backed bucket>` is less allocation-heavy than a `Map<number, Set<number>>` of boxed values. The moment the learner becomes allocation-dominant, it loses the fight in the browser. The literature supports the *idea* of the learner, but not a JS-specific data-structure theorem. [31]

Per-level policy selection without overfitting

With only 134 easy/successful training points and 3 pathological unsolved outliers, the safest selector discipline is not "more model." It is **smaller portfolio, fewer features, stronger abstention**. `SATzilla`-style pairwise cost-sensitive classification is a proven strong baseline, and `AutoFolio` explicitly treats pairwise classification, regression, and other selectors as choices inside a configurable selection stack. But the generalization literature also says the overfitting pressure grows with selector complexity, portfolio size, and the complexity of the algorithm-performance function itself. That matters a lot at your sample size. [32]

That is why I would not choose between pairwise forests, shallow boosting, and hand rules in the abstract. I would ship a **three-baseline bakeoff**: global default; a very small hand-curated decision list based on obvious signatures; and a simple local selector such as k-nearest-neighbours over a small feature set. There is actual precedent for simple local selectors outperforming more ornate portfolios on some SAT distributions, and the locality argument is especially attractive when the data set is small. If a learned selector cannot beat those baselines under leave-one-level-out or blocked CV, it should not route shipped levels. [33]

Portfolio design itself matters as much as the selector. Leyton-Brown et al.'s portfolio argument remains the cleanest statement: portfolios help when component solvers' easy inputs are **sufficiently uncorrelated**. Tiny weight perturbations of the same beam driver are therefore bad portfolio members; they create the illusion of diversity without actually generating selection signal. You want policy diversity at the mechanism level: different closure ordering, different must-cross scheduling, different portal-commitment handling, different pruning aggressiveness, maybe a distinct closer stage. The selector can only learn signal if the policies fail on different instances for different reasons. [34]

For the three hard levels, the correct published posture is **abstain to fallback**, not "trust the recommender to extrapolate." Online portfolio planners such as `Delfi` do per-instance routing, but even there the model predicts success under resource bounds among a known planner set. In your setting, the three hard levels are OOD by definition. The conservative deployment answer is therefore: low-confidence or OOD levels go to "default explorer + closer + exact micro-solver" rather than to a learned policy guess. [35]

Hint paths, solvability, and evaluation discipline

The hint paths are the best oracle you have

The hint paths should move from "unused asset" to "primary engineering oracle." First, they let you plot each heuristic component along a known-good trajectory and identify where ranking flattens or inverts. Second, they let you catch unsafe pruning immediately: if a new bound fires on a witness prefix, you have a concrete counterexample. Third, they give you supervised data for learning **rank corrections**, not only value regression. The recent planning literature has become more explicit on that last point: optimizing heuristic rankings for the target search procedure can matter more than regressing to `h*`, and imitation-style search learning uses oracle decisions precisely to reduce wavefront size and search effort. [6]

So for your concrete subquestions: plotting `h(s)` and each score term along the hint path is exactly the right move. The clean formalization is a **plateau-location-aware rank audit**: for each witness depth, record the rank of the witness successor among siblings or among beam candidates; then mark segments where witness depth decreases but rank does not improve. Those are not merely "bad values"; they are the exact

places where the search ordering is failing. If you ever decide to learn a corrective term, use ranking or imitation losses on those witness-derived comparisons before reaching for value regression or RL. The 2023 “rank, not estimate” paper is unusually aligned with this exact problem. [36]

On naming: “counterexample-guided bound tuning” is a reasonable description, but the closest canonical term in the literature is still CEGAR-style counterexample-guided refinement. The hint prefix that gets wrongly pruned is your counterexample; the fix is to refine the bound or state abstraction so that the same valid trace is not killed again. [24]

Solvability certificates and backward red-team

Because you already possess verified solution paths, the **cheapest certificate of solvability** is simply the hint path plus an independent validator. That is the exact dual of the unsolvability-certificate literature: when the answer is “solvable,” the certificate is a witness path; when the answer is “unsolvable,” the certificate is an inductive dead-state proof. In practical audit logs, that means you do not need a fancy theorem prover to justify “this level is solvable”; you need a deterministic replay validator that checks the packed path against the current semantics and emits a machine-checkable success report. [29]

The Eriksson–Helmert line is still useful, but for a different reason. It tells you what a certifying architecture looks like for *unsolvability*: compact proof objects, independent checking, polynomial verification where possible. It does **not** hand you an off-the-shelf complete backward operator for Pathfinder. In your puzzle, `reqInt` is the big gotcha because it is a global path property. A backward operator on `(tile_position, portal_state)` alone is not complete for a problem whose legality depends on remaining length, remaining intersections, and mandatory visits/crossings. If you want the recommended “30-minute red-team,” make it a backward or bidirectional checker on the **full product state that is suffix-Markovian**, not on a reduced state that forgets history-dependent legality. [37]

Heavy tails and what to report

Gomes, Selman, and Crato remain the right citation for the basic warning: combinatorial search runtimes are often heavy-tailed, and seed-level anecdotes are not enough. For your setup, I would still treat “30 seeds” as appropriate **for the outer randomized run**, but not as 30 fully independent algorithm observations if the internal attempt ladder reuses structure and policies. In other words, report the top-level seed as the unit of replication and treat attempt-level randomization as nested within that run. [38]

For claiming that a hard level is “solved,” report **both** a hit-rate number and a runtime number. The cleanest practical pair is: success rate over a fixed panel of distinct top-level seeds, and a capped runtime quantile such as median/p90 on the successful or all capped runs. For the stable 134-level set, replace “no level may flip at any seed” with a paired panel test: a small fixed seed panel for regression detection, plus the hard invariant that every embedded hint path still validates and no new prune kills a witness prefix. That removes the single-seed-outlier problem without multiplying audit cost by 30 across the whole catalogue. [39]

Ranked two-week investigation plan

1. **Build the hint-path oracle harness and make it a blocking CI gate.**
   This is the best leverage-per-hour move by a wide margin. It directly addresses your mass-regression

problem, your uncertainty about sign bugs, and your lack of per-level evidence when a “small fix” breaks 130+ levels. The harness should do four things immediately: replay every hint path under the current state model, assert that no prune fires on any hint prefix, record every score component along the hint, and emit witness-rank diagnostics for sibling states near the plateau depths. Literature support is strong from plan validation, certifying outputs, CEGAR-style counterexample use, ranking-based heuristic optimization, and imitation-style search diagnostics. [40]

2. **Add a ring-fenced closure stage for the L108 family, but make it bounded-cost and rank-aware, not plain BFS.**
   L108 is already telling you the explorer can reach the right neighbourhood; it just cannot discriminate inside it. The literature on final plateaus, tie-breaking, EES, and bounded-cost search makes this the most plausible small change that can convert a persistent failure into a stable solve without perturbing the whole solver. Keep the closer gated by explicit, logged predicates; keep depth around 6 by default; sort by a closure residual key rather than raw depth; and do not attempt MM first. This is the one change that is both literature-backed and realistically shippable by one senior engineer in two weeks. [41]

3. **Prototype an obligation-aware lower bound for L92-like stagnation using direct edge/transition landmarks or operator-counting, not a global scalar reweight.**
   L92 is not a closure problem. It is a residual-obligation accounting problem. The principled minimal intervention is to add an additional bound or ranking feature that charges unmet must-pass and must-cross obligations in a way that cannot invert globally because of one sign mistake. Encode must-crosses directly as transition/action landmarks; keep intersections as a separate lower-bound resource; and combine with a `max` /lexicographic policy or a small LP/IP relaxation rather than by adding another hand-tuned scalar. This is the change most likely to move the 18–24 lower-bound floor without detonating the easy set. [42]

4. **Separate telemetry from canonical results and treat rescue gates as calibrated classifiers.**
   You have already lost too much time to identity-stamp regressions and unfalsified gating proxies. This work is not glamorous, but it is what prevents the next two promising fixes from being reverted blindly. Move attempt telemetry to a side channel, log every gate predicate on every attempt, and evaluate historical precision/recall of the gate counters against known failure families. That gives you the data needed to replace unreachable thresholds with actual stall signals. It also pays off immediately for audit hygiene. [43]

5. **Defer full browser SMT/CP, full MM, and bare Luby restarts.**
   These are the tempting but low-ROI interventions under your constraints. Browser-side general-purpose exact solvers are real, but the current bundle-size and latency story is not compatible with your default ship target. Full meet-in-the-middle over a history-sensitive portal/intersection state is too much machinery for a two-week pilot. Luby alone does not have convincing evidence in your specific search regime without retained learning. All three can be revisited later; none should consume the next sprint. [44]

If capacity forces the list down to three items, the cut line should be exactly after the third item. Those three together attack the highest-probability win path for all three persistent failures while minimizing regression risk.

---

1 4 https://ai.dmi.unibas.ch/_files/teaching/hs22/po/slides/po-g03.pdf
https://ai.dmi.unibas.ch/_files/teaching/hs22/po/slides/po-g03.pdf

2 23 40 https://pureportal.strath.ac.uk/en/publications/vals-progress-the-automatic-validation-tool-for-pddl21-used-in-th
https://pureportal.strath.ac.uk/en/publications/vals-progress-the-automatic-validation-tool-for-pddl21-used-in-th

3 38 39 https://www.cs.cornell.edu/gomes/pdf/1997_gomes_cp_distributions.pdf
https://www.cs.cornell.edu/gomes/pdf/1997_gomes_cp_distributions.pdf

5 https://www.jair.org/index.php/jair/article/download/10667/25496/19843
https://www.jair.org/index.php/jair/article/download/10667/25496/19843

6 36 https://proceedings.neurips.cc/paper_files/paper/2023/file/50ea4dbd1cff6bd3daef939eff10c092-Paper-Conference.pdf
https://proceedings.neurips.cc/paper_files/paper/2023/file/50ea4dbd1cff6bd3daef939eff10c092-Paper-Conference.pdf

7 9 https://www.ijcai.org/Proceedings/09/Papers/288.pdf
https://www.ijcai.org/Proceedings/09/Papers/288.pdf

8 42 https://ai.dmi.unibas.ch/papers/pommerening-et-al-icaps2024.pdf
https://ai.dmi.unibas.ch/papers/pommerening-et-al-icaps2024.pdf

10 https://drops.dagstuhl.de/storage/01oasics/oasics-vol020-atmos2011/OASIcs.ATMOS.2011.64/OASIcs.ATMOS.2011.64.pdf
https://drops.dagstuhl.de/storage/01oasics/oasics-vol020-atmos2011/OASIcs.ATMOS.2011.64/OASIcs.ATMOS.2011.64.pdf

11 https://ai.dmi.unibas.ch/_files/teaching/hs25/po/slides/po-f11-handout.pdf
https://ai.dmi.unibas.ch/_files/teaching/hs25/po/slides/po-f11-handout.pdf

12 41 https://cdn.aaai.org/ojs/10071/10071-13-13599-1-2-20201228.pdf
https://cdn.aaai.org/ojs/10071/10071-13-13599-1-2-20201228.pdf

13 https://www.cs.toronto.edu/~sheila/2542/s14/A1/hoffmannebel-FF-jair01.pdf
https://www.cs.toronto.edu/~sheila/2542/s14/A1/hoffmannebel-FF-jair01.pdf

14 15 https://www.cs.unh.edu/~ruml/papers/ees-ijcai-11.pdf
https://www.cs.unh.edu/~ruml/papers/ees-ijcai-11.pdf

16 https://cdn.aaai.org/ojs/21256/21256-13-25269-1-2-20220628.pdf
https://cdn.aaai.org/ojs/21256/21256-13-25269-1-2-20220628.pdf

17 https://www.cs.toronto.edu/~sheila/publications/bai-mci-icaps06.pdf
https://www.cs.toronto.edu/~sheila/publications/bai-mci-icaps06.pdf

18 19 https://ai.dmi.unibas.ch/research/reading_group/holte-et-al-aaai2016.pdf
https://ai.dmi.unibas.ch/research/reading_group/holte-et-al-aaai2016.pdf

20 22 https://erichorvitz.com/drestart.pdf
https://erichorvitz.com/drestart.pdf

21 https://www.cs.ubc.ca/labs/algorithms/Projects/Ablation/papers/FawcettHoos-joh2016-ablationAnalysis.pdf
https://www.cs.ubc.ca/labs/algorithms/Projects/Ablation/papers/FawcettHoos-joh2016-ablationAnalysis.pdf

24 https://ai.dmi.unibas.ch/papers/seipp-helmert-jair2018.pdf
https://ai.dmi.unibas.ch/papers/seipp-helmert-jair2018.pdf

25 https://d-krupke.github.io/cpsat-primer/05_parameters.html
https://d-krupke.github.io/cpsat-primer/05_parameters.html

26 43 https://issues.fast-downward.org/issue589
https://issues.fast-downward.org/issue589

27 https://microsoft.github.io/z3guide/programming/Z3%20JavaScript%20Examples
https://microsoft.github.io/z3guide/programming/Z3%20JavaScript%20Examples

28 https://iris.unical.it/handle/20.500.11770/376938
https://iris.unical.it/handle/20.500.11770/376938

29 https://ai.dmi.unibas.ch/papers/eriksson-et-al-icaps2017.pdf
https://ai.dmi.unibas.ch/papers/eriksson-et-al-icaps2017.pdf

30 31 https://ml.rwth-aachen.de/~hector.geffner/www.dtic.upf.edu/~hgeffner/Nir-ICAPS-2016.pdf
https://ml.rwth-aachen.de/~hector.geffner/www.dtic.upf.edu/~hgeffner/Nir-ICAPS-2016.pdf

32 https://www.cs.ubc.ca/labs/algorithms/Projects/SATzilla/SAT12-EvaluatingSolverContributions.pdf
https://www.cs.ubc.ca/labs/algorithms/Projects/SATzilla/SAT12-EvaluatingSolverContributions.pdf

33 https://poincare.matf.bg.ac.rs/~janicic/papers/2011-air-argosmart.pdf
https://poincare.matf.bg.ac.rs/~janicic/papers/2011-air-argosmart.pdf

34 https://www.cs.ubc.ca/~kevinlb/papers/portfolio-IJCAI.pdf
https://www.cs.ubc.ca/~kevinlb/papers/portfolio-IJCAI.pdf

35 https://ipc2018-classical.bitbucket.io/planner-abstracts/teams_23_24.pdf
https://ipc2018-classical.bitbucket.io/planner-abstracts/teams_23_24.pdf

37 https://ai.dmi.unibas.ch/papers/eriksson-helmert-icaps2020.pdf
https://ai.dmi.unibas.ch/papers/eriksson-helmert-icaps2020.pdf

44 https://github.com/cpitclaudel/z3.wasm
https://github.com/cpitclaudel/z3.wasm